

Building with VelocityGrid

A practical, example-first walkthrough of the core grid, VelocityGridExt, server-side data (SSRM), live DataProviders, and the storage/config layers underneath. Every snippet on this page is pulled from real, working code in the repo — nothing here is invented.

Quickstart

A grid on screen in one file

CSRM

In-memory row data, the default mode

VelocityGridExt

Ribbon, alerts, smart edit, renderers

SSRM

Server-driven data at any scale

Storage & Config

Persistence underneath everything

Import Map

Where does X live now?

Install & Quickstart

One package gets you a working grid. Everything else in this guide is additive.

npm

```
npm install @wellsfargo-starui/velocity-grid
```

index.ts — a complete, working grid

```
import { VelocityGrid } from '@wellsfargo-starui/velocity-grid';
import '@wellsfargo-starui/velocity-grid/style.css';

const grid = new VelocityGrid<{ id: string; name: string; value: number }>(
  document.getElementById('grid')!,
  {
    columnDefs: [
      { field: 'id', headerName: 'ID', pinned: 'left', width: 100 },
      { field: 'name', headerName: 'Name', flex: 1 },
      { field: 'value', headerName: 'Value', type: 'number', width: 120, aggFunc: 'sum' }
    ],
    getRowId: (row) => row.id,
    rowSelection: 'multiple',
    theme: 'vg-theme-quartz',
  },
);

grid.setRowData([{ id: 'a', name: 'Apple', value: 12.5 }]);
```

THE ONE REQUIRED FIELD

Beyond a container element and `columnDefs`, `getRowId` is the one option the constructor actually enforces — the grid throws if it's missing. Everything else has a sane default.

Creating a Grid

The `VelocityGrid` class is the engine every other layer in this guide builds on top of. Construct it with a container element and an options object.

```
new VelocityGrid<TRow>(container: HTMLElement, options: VelocityGridOptions<TRow>)
```

A slightly more realistic example, with row selection and a dark theme:

```
import { VelocityGrid } from '@wellsfargo-starui/velocity-grid';
import type { CColDef } from '@wellsfargo-starui/velocity-grid';

const grid = new VelocityGrid<Position>(gridHost, {
  getRowId: (r) => r.id,
  columnDefs: COLUMNS,
  theme: 'vg-theme-cursor-dark',
  density: 'normal',
});

grid.on('gridReady', () => {
  grid.setRowData(makeRows(60));
});

grid.on('cellClicked', (e) => console.log(e));
```

Column Definitions

Column defs are plain objects. The fields you'll reach for most:

FIELD	WHAT IT DOES
<code>field</code>	Which row property this column reads
<code>headerName</code>	Display label
<code>width</code> / <code>flex</code> / <code>minWidth</code> / <code>maxWidth</code>	Sizing — fixed vs. proportional
<code>pinned</code>	<code>'left'</code> or <code>'right'</code>
<code>cellDataType</code>	<code>'text'</code> or <code>'number'</code> — drives default formatting/filtering
<code>valueGetter</code> / <code>valueFormatter</code>	Derive or format a display value
<code>cellRenderer</code> / <code>cellRendererParams</code>	Custom cell painting — see Cell Renderers
<code>filter</code> / <code>filterParams</code>	<code>'text'</code> · <code>'number'</code> · <code>'date'</code> · <code>'set'</code>
<code>sortable</code> / <code>resizable</code>	Column-level toggles
<code>editable</code> / <code>cellEditor</code>	Inline editing
<code>aggFunc</code>	Aggregate function when grouped — <code>'sum'</code> , <code>'avg'</code> , an array, or custom

Real examples, from a live positions blotter:

```
{
  field: 'cusip', headerName: 'CUSIP', width: 110,
  editable: true,
  filter: 'text',
  filterParams: { caseSensitive: false, trimInput: true },
},
{
  field: 'ticker', headerName: 'Ticker', width: 100, editable: true,
  cellEditor: 'select',
  cellEditorParams: { values: ['AAPL', 'MSFT', 'GOOG', 'AMZN', 'NVDA'] },
  filter: 'set',
},
{
  field: 'notionalAmount', headerName: 'Notional', type: 'money',
  width: 130, aggFunc: 'sum', editable: true,
  cellEditor: 'number',
  cellEditorParams: { min: 0, precision: 2 },
  filter: 'number',
},
}
```

`type: 'money'` above resolves through a named `columnTypes` entry declared once on the grid's own options (`columnTypes: {'{'} money: {'{'} cellDataType: 'number', valueFormatter: ... {'{'} {'{'}}`) — define a shape once, reuse it on every column that needs it.

CSRM — Client-Side Row Model

CSRM is the default. You don't opt into it — you opt *out* of it by switching to SSRM. If you pass `rowData` (at construction or later via `grid.setRowData()`) and don't set `rowModelType: 'serverSide'`, you're on CSRM: the full dataset lives in memory and the grid's worker handles sort/filter/group over it.

CSRM

```
rowModelType: 'clientSide' (default)
```

- You hand the grid the full row set up front
- Sort / filter / group run entirely in-browser (off the main thread, in a worker)
- Simplest mental model — best default for anything that comfortably fits in memory

SSRM

```
rowModelType: 'serverSide'
```

- The grid asks *you* for windows of rows as the user scrolls/sorts/groups
- Needed for datasets too large to hold client-side, or truly live/server-driven data
- See **SSRM & Data Providers** below

GOOD TO KNOW

`setRowData()` is a CSRM-only call — it's a no-op with a console warning once the grid is in SSRM mode. Feed SSRM grids through the datasource callback instead, never `setRowData`.

Grid API

The instance you get back from `new VelocityGrid(...)` is the API surface you'll use for everything after construction. The methods you'll reach for most:

METHOD	PURPOSE
<code>setRowData(rows)</code>	Full CSRM data replace
<code>applyTransaction(tx)</code>	Sync add / update / remove
<code>applyTransactionAsync(tx)</code>	Batched, throttled add / update / remove — for high-frequency ticks
<code>getSelectedRowIds()</code> / <code>setSelectedRowIds(ids)</code>	Selection
<code>updateGridOptions(partial)</code>	Patch multiple options at once (e.g. new <code>columnDefs</code>)
<code>setGridOption(key, value)</code> / <code>getGridOption(key)</code>	Patch or read a single runtime option — see Grid Runtime Options
<code>sizeColumnsToFit()</code> / <code>autoSizeAllColumns()</code>	Distribute or autosize column widths
<code>setColumnsVisible(keys, visible)</code> / <code>setColumnsPinned(keys, side)</code>	Column visibility & pinning
<code>getColumnState()</code> / <code>applyColumnState(state)</code> / <code>resetColumnState()</code>	Save & restore the whole column layout
<code>setSortModel(model)</code> / <code>getSortModel()</code>	Sort state
<code>setFilterModel(model)</code> / <code>getFilterModel()</code>	Filter state
<code>setGroupModel({'{' rowGroupCols {''}}')</code>	Row grouping — one or more levels
<code>exportDataAsCsv(params)</code> / <code>getDataAsCsv(params)</code>	CSV export — download or return a string
<code>on(type, handler)</code> / <code>off(...)</code>	Events — <code>gridReady</code> , <code>firstDataRendered</code> , <code>cellClicked</code> , <code>modelUpdated</code> , <code>selectionChanged</code> , ...
<code>getDisplayedRowCount()</code>	Row count after the current filter/sort
<code>destroy()</code>	Tear down the grid, listeners, and worker

Theming

Import the stylesheet once, then pick a theme via the `theme` option — the grid stamps the corresponding class on its own root element, so there's nothing to add to your container by hand.

```
import '@wellsfargo-starui/velocity-grid/style.css';

const grid = new VelocityGrid(host, {
  theme: 'vg-theme-cursor-dark',
  // ...
});

// swap it later at runtime:
grid.setTheme('vg-theme-quartz-dark');
grid.setDensity('compact'); // 'compact' | 'normal' | 'comfortable'

// per-token overrides, on top of whichever theme is active:
grid.setThemeParams({ '--vg-row-height': '32px' });
```

Built-in themes: `vg-theme-quartz` / `-dark` (the default light theme), `vg-theme-starui` / `-dark`, `vg-theme-cursor` / `-dark`, and `vg-theme-auto`, which follows the OS color scheme automatically.

Sort, Filter, Group

Per-column, these are just fields on the column def:

```
{ field: 'cusip', sortable: true, filter: 'text', filterParams: { caseSensitive: false } },
{ field: 'notionalAmount', filter: 'number', filterParams: { maxNumConditions: 2 } },
{ field: 'ticker', filter: 'set' },
```

A floating filter row across the whole grid: `floatingFilter: true` on the grid options (or on `defaultColDef` to apply it everywhere at once). Grouping is enabled per column with `enableRowGroup: true`, then activated on demand:

```
grid.setGroupModel({ rowGroupCols: ['ticker', 'sector'] }); // multi-level
grid.setGridOption('groupDisplayType', 'multipleColumns'); // one auto-group column per level
```

Wiring It Up

VelocityGridExt is the spreadsheet-power-user layer — a ribbon toolbar, alerts, bulk editing, conditional styling, calculated columns, and more, all on top of a core grid it manages for you.

YOU DON'T CONSTRUCT VELOCITYGRID YOURSELF

`VelocityGridExt` builds and owns its own `VelocityGrid` internally. You get at it afterward via `ext.grid` — that's what you pass to feature engines like the calc or rules wiring functions below.

A complete, working setup


```

import {
  VelocityGridExt,
  titleBarExtensions,
  ribbonExtensions,
} from '@wellsfargo-starui/velocity-grid-ext';
import { wireIntoKernel as wireFormat } from '@wellsfargo-starui/velocity-grid/format';
import { wireIntoKernel as wireCalc } from '@wellsfargo-starui/velocity-grid/calc';
import { wireIntoKernel as wireRules } from '@wellsfargo-starui/velocity-grid/rules';
import { wireEditIntoKernel } from '@wellsfargo-starui/velocity-grid-ext/edit';
import '@wellsfargo-starui/velocity-grid/style.css';

let editHandle: ReturnType<typeof wireEditIntoKernel> | undefined;

const ext = new VelocityGridExt<PositionRow>(host, {
  gridId: 'positions',
  getRowId: (row) => row.id,
  columnDefs: COLUMN_DEFS,
  defaultColDef: { resizable: true, sortable: true, editable: true, minWidth: 80 },
  theme: 'vg-theme-quartz',
  sideBar: { toolPanels: ['columns', 'filters'] },
  rowData: seedRows(),
  ext: {
    extensions: [
      ...titleBarExtensions({ name: 'Positions' }),
      ...ribbonExtensions({ edit: () => editHandle }),
    ],
  },
});

// feature engines are wired in AFTER construction, against ext.grid:
wireFormat(ext.grid);
editHandle = wireEditIntoKernel(ext.grid);
wireCalc(ext.grid);
wireRules(ext.grid);

```

Modules

Unlike feature engines, the settings-sheet modules below are **on by default** — there's no per-module boolean flag. They register automatically and show up as entries behind the  launcher. The only lever is `ext.extensions`, which lets you remove or replace any of them by id (e.g. `{{ remove: 'settings-launcher' }}`).

alertsModule

Trigger / severity / message / channel rule editor for row-level alerts.

bulkUpdateModule

Settings for the Bulk Update edit operation, linked to change history.

calculatedColumnsModule

A CodeMirror expression editor for defining calculated columns.

columnSettingsModule

Per-column overrides — aggregation, flags — with draft/Save/Reset.

conditionalStylingModule

Rule-based conditional cell styling with an expression editor.

dataChangeHistoryModule

Settings plus a live journal of every edit made to the grid.

plusMinusModule

Configurable $\pm$ quick-adjust buttons and their nudge amounts.

smartEditModule

Global / Operations / Safety settings for $\times$, $\div$, $+$, $-$ and set-value edits.

shortcutsModule

Letter-key shortcuts for numeric ops, editable per shortcut.

`expressionLab` exists but is *not* in the default bundle — add it explicitly via `ext.extensions` if you want it.

Ribbon Toolbar

The ribbon host exists in the DOM either way, but stays empty and hidden until you opt in with `ribbonExtensions()`, as shown in the setup example above. No separate container is required — it composes straight into `ext.extensions`.

```
ext: {  
  extensions: [  
    ...ribbonExtensions({ edit: () => editHandle }),  
  ],  
}
```

The `edit` field is a lazy getter because `wireEditIntoKernel` typically runs right after the `VelocityGridExt` constructor, not before it — see [Edit Engine](#).

Cell Renderers

Around 40 named cell renderers — sparklines, bars, badges, tick-flash numeric cells, and more — ship under the `/renderers` subpath. Wire them in once, then reference them by name from any column def, or use the bundled builder helper:

```
import { wireRenderersIntoKernel } from '@wellsfargo-starui/velocity-grid-ext/renderers';

const renderers = wireRenderersIntoKernel(grid);

// via the builder:
cols.push({
  ...renderers.colDef.renderer('price', 'price', { prevField: 'prevPrice' }, { colId:
'price' }),
  headerName: 'Price',
  width: 110,
});

// or write the plain column def directly, once wireRenderersIntoKernel has run:
{ field: 'price', cellRenderer: 'price', cellRendererParams: { prevField: 'prevPrice' } }
```

Edit Engine

Smart Edit, Bulk Update, and the $\pm$ nudge buttons are all wired in with one call:

```
import { wireEditIntoKernel } from '@wellsfargo-starui/velocity-grid-ext/edit';

const editHandle = wireEditIntoKernel(ext.grid, {
  nudges: [{ label: '+1bp', delta: 0.0001 }],
  shortcuts: [{ key: 'x', op: 'multiply', amount: 2 }],
});

// later:
editHandle.smartEdit.preview(...);
editHandle.smartEdit.apply(...);
editHandle.journal;           // change history
editHandle.getSettings();     // current settings snapshot
```

History, Smart Edit, Bulk Update, $\pm$ nudges, and shortcuts are all `enabled: true` by default — pass a `settings` override only for the ones you want to change.

Teardown

```
ext.destroy();
```

One call cascades all the way down: extensions, the ribbon shell, and the underlying grid are all disposed. The edit engine specifically listens for the grid's own `gridPreDestroyed` event, so you don't need to separately call `editHandle.destroy()` — it happens

automatically.

Switching to SSRM

Server-Side Row Model trades "the grid holds everything in memory" for "the grid asks you for windows of rows as needed" — the right call once a dataset is too large to hold client-side, or is genuinely live and server-driven.

```
rowModelType: 'serverSide',  
serverSideDatasource: myDatasource,    // an object implementing getRows(params)  
cacheBlockSize: 100,                  // rows fetched per block, default 100
```

A minimal datasource just needs a `getRows` method that calls back with a page of rows:

```
interface IServerSideDatasource<TRow> {  
    getRows(params: IServerSideGetRowsParams<TRow>): void;  
    destroy?(): void;  
}  
// inside getRows:  
params.success({ rowData: page, rowCount: totalIfKnown });  
// or, on failure:  
params.fail();
```

In practice, you won't usually hand-write one of these — you'll get a ready-made datasource from a `DataProvider`, as below.

A complete, working SSRM grid backed by a live feed

```

import { VelocityGrid } from '@wellsfargo-starui/velocity-grid';
import { StompPerspectiveProvider } from '@wellsfargo-starui/velocity-grid-perspective';
import '@wellsfargo-starui/velocity-grid/style.css';

const provider = new StompPerspectiveProvider({
  feed: 'stomp',
  wsUrl: 'ws://localhost:8082',
  snapshotRows: 5000,
  rate: 40,
  batchSize: 200,
});

const grid = new VelocityGrid(document.getElementById('grid')!, {
  theme: 'vg-theme-cursor-dark',
  rowSelection: 'multiple',
  ...provider.gridOptions(), // spread FIRST – brings rowModelType,
  serverSideDatasource, etc.
});

const detach = provider.attach(grid); // live ticks + sort/filter/group sync, going
forward

```

`provider.gridOptions()` hands back the whole SSRM bundle in one object — `getRowId`, `columnDefs`, `rowModelType`: `'serverSide'`, `serverSideDatasource: this`, sensible cache-block and concurrency defaults, and a pinned grand total row. Spread it first so any options you pass afterward can still override it.

The DataProvider Contract

Every `DataProvider` in `@wellsfargo-starui/velocity-grid-data` implements the same interface, regardless of transport:

```

interface IDataProvider<T = Record<string, unknown>> {
  readonly providerId: string;
  start(): Promise<void>;
  stop(): Promise<void>;
  refresh(): Promise<void>;
  getData(): readonly T[];
  getConfig(): DataProviderConfig;
  getStatus(): ProviderStatus;
  onRowsReceived(handler): Unsubscribe;
  onTick(handler): Unsubscribe;
  onError(handler): Unsubscribe;
  onStatus(handler): Unsubscribe;
  destroy(): void;
}

```

A REAL GOTCHA, WORTH KNOWING UP FRONT

The DataProvider layer's own `rowModel: 'serverSide'` flag is documented as effectively ignored — that path feeds `setRowData` / `applyTransaction` like CSRM regardless of what you set it to. For genuine SSRM, go through the [Perspective bridge](#) below, not the DataProvider's own row model flag.

STOMP Live Feeds

The catalog-level transport config — what you'd hand to the [provider config editor](#) or store in a saved connection — looks like this:

```
{
  providerType: 'stomp',
  config: {
    websocketUrl: 'ws://localhost:8082',
    listenerTopic: '/snapshot/positions/TRADER001',
    requestMessage: '/snapshot/positions/TRADER001/40/200',
    snapshotEndToken: 'Success',
    keyColumn: 'positionId',
    heartbeat: { outgoing: 4000, incoming: 4000 },
    columnDefinitions: EDITOR_COLUMNS,
  },
}
```

TWO DIFFERENT FIELD-NAME SETS, BY DESIGN

The catalog/transport config above uses `websocketUrl` / `listenerTopic` / `requestMessage`. `StompPerspectiveProvider`'s own constructor — shown in the SSRM example above — uses `wsUrl` / `snapshotTopic` / `triggerTopic` instead, because it runs an independent STOMP client. The bridge function below performs that rename for you automatically; you shouldn't need to hand-translate between them.

Connecting to Perspective

To drive SSRM from a catalog-shaped provider config rather than hand-building a `StompPerspectiveProvider` config yourself, convert it first:

```
import { dataProviderConfigToPerspective } from '@wellsfargo-starui/velocity-grid-perspective';

const mapped = dataProviderConfigToPerspective(catalogEntry); // field-renamed, ready to use
const provider = new StompPerspectiveProvider(mapped);
const gridOptions = provider.gridOptions();
```

For an app managing multiple saved connections, `PerspectiveDataProviderController` wraps this whole flow — catalog, active-provider switching, and telemetry — behind a few calls:

```
import { LocalStorageConfigBackend, registerDefaultTransports } from '@wellsfargo-starui/velocity-grid-data';
import { PerspectiveDataProviderController } from '@wellsfargo-starui/velocity-grid-perspective';

registerDefaultTransports();
const catalog = new LocalStorageConfigBackend({ storage });
const dataController = new PerspectiveDataProviderController({ catalog, onTelemetry, onActiveChange });

await catalog.save(providerConfig);
await dataController.setActiveProvider(providerConfig.providerId);
```


Storage

The lowest layer — a small, swappable key/value transport that everything above it persists through.

```
interface IStorage {
  getItem(key: string): string | null | Promise<string | null>;
  setItem(key: string, value: string): void | Promise<void>;
  removeItem(key: string): void | Promise<void>;
}
```

Three implementations ship out of the box — pick whichever matches where you want data to live:

CLASS	BACKS ONTO
LocalStorage	Browser <code>localStorage</code>
MemoryStore	An in-memory <code>Map</code> — gone on reload
RestStore	An HTTP key/value API (<code>{{baseUrl}}/kv/{{key}}</code>)

```
import { LocalStore, storageGet, storageSet } from '@wellsfargo-starui/velocity-grid-data/storage';

const store = new LocalStore({ onError: (op, err, key) => console.warn(op, key, err) });
await storageSet(store, 'my-key', JSON.stringify({ foo: 1 }));
const raw = await storageGet(store, 'my-key'); // string | null
```

AppData

A named key/value bag, one layer up from raw Storage — built for resolving `{{providerName.key}}` template tokens inside grid/provider configs.

```
import { PersistedAppDataStore } from '@wellsfargo-starui/velocity-grid-data/appdata';

const appData = new PersistedAppDataStore('default'); // backed by LocalStore() by default
appData.set('user', 'theme', 'dark');
const theme = appData.get('user', 'theme'); // 'dark'
```

Swap the backend the same way you would for Storage on its own:

```
import { RestStore } from '@wellsfargo-starui/velocity-grid-data/storage';

const remote = new PersistedAppDataStore('default', { storage: new RestStore({ baseUrl:
'/api' }) });
```

AppData

{{name.key}} template resolution

Provider Config

saved DataProvider connections

↓ both persist through ↓

Storage (IStorage)

LocalStore · MemoryStore · RestStore

AppData and the provider config catalog are independent domain layers that both happen to sit on Storage directly — AppData isn't built *on top of* the provider catalog, or vice versa.

Provider Config Editor

A full catalog UI ships with the data package — list, form, JSON import/export, diagnostics. The way most apps actually use it is a single call that opens it in a detached popout:

```
import { openProviderEditorPopout } from '@wellsfargo-starui/velocity-grid-data/editor';

const handle = openProviderEditorPopout({
  backend: catalog,
  providerId,           // focus a specific saved provider, or null
  themeSource,          // an element carrying your .vg-theme-* class, for popout
  theming
  onSave: () => refresh(),
  onClose: () => refresh(),
});
if (!handle) { /* popup was blocked */ }
```

Or mount it directly in your own layout:

```
import { mountDataProviderEditor } from '@wellsfargo-starui/velocity-grid-data/editor';

const shell = mountDataProviderEditor({ mount: containerEl, backend, initialProviderId:
'stomp-1' });
// shell.destroy() to unmount
```

IMPORT VALIDATION

Importing a saved config file is validated before anything touches the catalog: an unrecognized `providerType` is rejected, required fields for that transport must be present (e.g. `websocketUrl` + `listenerTopic` for STOMP), and a STOMP `websocketUrl` specifically must parse as a real `ws://` / `wss://` URL. Nothing gets auto-saved on import either way — a bad or unreviewed import just loads into the form for you to check before hitting Save.

Grid Runtime Options

Separate from provider config — this is changing the grid's own options after it's already built.

```
grid.setGridOption('enableCellChangeFlash', false);
// ...
grid.setGridOption('enableCellChangeFlash', true);

grid.updateGridOptions({ columnDefs: nextColumnDefs }); // batched, for multi-option or
columnDefs changes
```

`setGridOption` throws if the key is one of a handful of options that can only be set at construction time — `columnDefs`, `getRowId`, and `worker` among them. Route those through `updateGridOptions` instead.

Import Map

Fourteen packages were recently consolidated into four. Every import path below is current — if you're holding onto an example that imports from something like `@wellsfargo-starui/velocity-grid-calc` directly, it's stale; that's now a subpath.

I WANT TO USE...	IMPORT FROM
The core grid	<code>@wellsfargo-starui/velocity-grid</code>
Calculated columns / aggregation engine	<code>@wellsfargo-starui/velocity-grid/calc</code>
Expression parser / editor	<code>@wellsfargo-starui/velocity-grid/expression</code> , <code>/expression/editor</code>
Cell/number formatting DSL	<code>@wellsfargo-starui/velocity-grid/format</code>
Alerts & conditional rules engine	<code>@wellsfargo-starui/velocity-grid/rules</code>
VelocityGridExt itself (ribbon, modules)	<code>@wellsfargo-starui/velocity-grid-ext</code>
Smart Edit / Bulk Update / ± nudges	<code>@wellsfargo-starui/velocity-grid-ext/edit</code>
The ~40 cell renderers	<code>@wellsfargo-starui/velocity-grid-ext/renderers</code>
CSV / export helpers	<code>@wellsfargo-starui/velocity-grid-ext/export</code>
The customizer web components	<code>@wellsfargo-starui/velocity-grid-ext/customizer</code>
DataProviders, provider config editor	<code>@wellsfargo-starui/velocity-grid-data</code> , <code>/editor</code>
Storage (LocalStorage / RestStore / MemoryStore)	<code>@wellsfargo-starui/velocity-grid-data/storage</code>
AppData	<code>@wellsfargo-starui/velocity-grid-data/appdata</code>
Perspective / SSRM bridge	<code>@wellsfargo-starui/velocity-grid-perspective</code>
All of the above, one dependency	<code>@wellsfargo-starui/velocity-grid-all</code> (meta-package — see note below)

ABOUT THE META-PACKAGE

`velocity-grid-all` is ready to use the moment these packages are published somewhere npm can fetch from — it isn't installable on its own from a local tarball today. For a one-command local install straight from packed tarballs, use `scripts/install-tarballs-into.mjs` in the repo instead.

Every code sample on this page was pulled from real source in the repo — demo apps, package READMEs, and the actual TypeScript types — verified against the current state of the codebase, not written from memory.